

Day2-Case Study 1: Online Shopping Order System

Collection: order

```
_id: ObjectId('6a117dcbc043d79056b637af')
customer_id: "C101"
name: "Asha"
▼ items: Array (2)
  ▼ 0: Object
    product: "Mobile"
    price: 15000
  ▼ 1: Object
    product: "Earphones"
    price: 2000

_id: ObjectId('6a117dcbc043d79056b637b0')
customer_id: "C102"
name: "Kiran"
▼ items: Array (2)
  ▼ 0: Object
    product: "Mobile"
    price: 15000
  ▼ 1: Object
    product: "Laptop"
    price: 50000

_id: ObjectId('6a117dcbc043d79056b637b1')
customer_id: "C103"
name: "Neha"
▼ items: Array (2)
  ▼ 0: Object
    product: "Earphones"
    price: 2000
  ▼ 1: Object
    product: "Laptop"
    price: 50000
```

SCENARIO 1: Mobile Product Revenue Report

Scenario

The company wants a report of **Mobile product orders** with these conditions:

Filters:

✓ Product must be Mobile ✓ Price must be ≥ 15000

They want:

- ✓ Total customers who bought Mobile ✓ Total revenue
- ✓ Average, minimum, maximum price
- ✓ List of customer names
- ✓ First customer who ordered

Store result in collection: mobile_report

```

> db.order.aggregate([
  {$unwind:"$items"},
  {$match:{"items.product": "Mobile", "items.price": { $gte: 15000 } } },
  { $group: {
    _id: "$items.product",
    totalCustomers:{$sum:1},
    totalRevenue:{$sum:"$items.price"},
    averagePrice:{$avg:"$items.price"},
    minimumPrice:{$min:"$items.price"},
    maximumPrice:{$max:"$items.price"},
    customerNames:{$push:"$name"},
    firstCustomer:{$first:"$name"}
  }},{ $out:"mobile_report"}
])
<
> db.mobile_report.find()
< {
  _id: 'Mobile',
  totalCustomers: 2,
  totalRevenue: 30000,
  averagePrice: 15000,
  minimumPrice: 15000,
  maximumPrice: 15000,
  customerNames: [
    'Asha',
    'Kiran'
  ],
  firstCustomer: 'Asha'
}

```

SCENARIO 2: High Value Products Report

Scenario

The management wants a report of products where:

Filters:

✓ Price >= 15000 ✓ Product = Mobile or Laptop

They want:

✓ Number of orders per product

✓ Total revenue per product

✓ Average, min, max price

✓ Customer list per product

Save output into: high_value_products_report

```

> db.order.aggregate([
  {$unwind:"$items"},
  {$match:{"items.price":{$gte:15000},"items.product":{$in:["Mobile","Laptop"]}}},
  {$group:{
    _id:"$items.product",
    numberOfOrders:{$sum:1},
    totalRevenue:{$sum:"$items.price"},
    averagePrice:{$avg:"$items.price"},
    minimumPrice:{$min:"$items.price"},
    maximumPrice:{$max:"$items.price"},
    customerList:{$push:"$name"}
  }},{$out:"high_value_products_report"}
])
<
> db.high_value_products_report.find()
< {
  _id: 'Mobile',
  numberOfOrders: 2,
  totalRevenue: 30000,
  averagePrice: 15000,
  minimumPrice: 15000,
  maximumPrice: 15000,
  customerList: [
    'Asha',
    'Kiran'
  ]
}
{
  _id: 'Laptop',
  numberOfOrders: 2,
  totalRevenue: 100000,
  averagePrice: 50000,
  minimumPrice: 50000,
  maximumPrice: 50000,
  customerList: [
    'Kiran',
    'Neha'
  ]
}

```

SCENARIO 3: Customer-wise Spending Report

Scenario

The company wants a report of customers who:

Filters:

✓ Have purchased more than one product ✓ Total amount spent $\geq$ 30000

They want:

✓ Customer name
 ✓ Total amount spent

- ✓ Average price
- ✓ Minimum and maximum product price
- ✓ List of products purchased

Save output into: customer_spending_report

```
> db.order.aggregate([
  {$unwind:"$items"},
  {$group:{
    _id:"$name",totalSpent:{$sum:"$items.price"},averagePrice:{$avg:"$items.price"},minimumPrice:{$min:"$items.price"},
    maximumPrice:{$max:"$items.price"},productList:{$push:"$items.product"},productCount:{$sum:1}
  }},
  {$match:{productCount:{$gt:1},totalSpent:{$gte:30000}}},
  {$project:{customerName: "$_id",totalSpent: 1,averagePrice: 1,minimumPrice: 1,maximumPrice: 1,productList: 1,_id: 0}},
  { $out: "customer_spending_report" }
])
<
```

```
> db.customer_spending_report.find()
< {
  _id: ObjectId('6a118a5f4ef97f1225d8746a'),
  totalSpent: 65000,
  averagePrice: 32500,
  minimumPrice: 15000,
  maximumPrice: 50000,
  productList: [
    'Mobile',
    'Laptop'
  ],
  customerName: 'Kiran'
}
{
  _id: ObjectId('6a118a5f4ef97f1225d8746b'),
  totalSpent: 52000,
  averagePrice: 26000,
  minimumPrice: 2000,
  maximumPrice: 50000,
  productList: [
    'Earphones',
    'Laptop'
  ],
  customerName: 'Neha'
}
```

Case Study 2 : Movie Ticket Booking System

Table 1: CUSTOMERS

```
< {
  _id: 1,
  name: 'Arun',
  profile: {
    age: 25,
    city: 'Chennai'
  }
}
{
  _id: 2,
  name: 'Priya',
  profile: {
    age: 22,
    city: 'Bangalore'
  }
}
{
  _id: 3,
  name: 'Rahul',
  profile: {
    age: 30,
    city: 'Hyderabad'
  }
}
{
  _id: 4,
  name: 'Neha',
  profile: {
    age: 28,
    city: 'Mumbai'
  }
}
{
  _id: 5,
  name: 'Kiran',
  profile: {
    age: 35,
    city: 'Pune'
  }
}
}
```

Table 2: Movies

🏠 movies

	_id String	title String	genre String	price Int32
1	"M1"	"Inception"	"Sci-Fi"	300
2	"M2"	"Avatar"	"Fantasy"	350
3	"M3"	"Interstellar"	"Sci-Fi"	400

Table 3:BOOKINGS

🏠 bookings				
	_id Int32	customer_id Int32	movie_id String	show_date String
1	301	1	"M1"	"2026-02-01"
2	302	1	"M2"	"2026-02-03"
3	303	2	"M1"	"2026-02-05"
4	304	3	"M3"	"2026-02-06"
5	305	3	"M1"	"2026-02-08"
6	306	4	"M2"	"2026-02-10"
7	307	5	"M3"	"2026-02-12"
8	308	5	"M2"	"2026-02-14"

Query 1: Total bookings per movie

Goal:

Find how many times each movie was booked.

```
> db.bookings.aggregate([
  { $group: {
    _id: "$movie_id",
    totalBookings: { $sum: 1 }
  }},
  { $sort: { totalBookings: -1 } }
])
< {
  _id: 'M1',
  totalBookings: 3
}
{
  _id: 'M2',
  totalBookings: 3
}
{
  _id: 'M3',
  totalBookings: 2
}
```

Query 2: Movie-wise revenue report

Goal:

Find revenue and price statistics for each movie.

Result:

For each movie, display:

- total tickets sold
- total revenue
- average ticket price
- minimum .and maximum ticket price

```
> db.bookings.aggregate([
  {$lookup:{
    from: "movies",
    localField: "movie_id",
    foreignField: "_id",
    as: "movieDetails"
  }},
  { $unwind: "$movieDetails" },
  { $group: {
    _id: "$movie_id",
    totalTicketsSold: { $sum: 1 },
    totalRevenue: { $sum: "$movieDetails.price" },
    averagePrice: { $avg: "$movieDetails.price" },
    minimumPrice: { $min: "$movieDetails.price" },
    maximumPrice: { $max: "$movieDetails.price" }
  }},
  {$sort:{totalRevenue:-1}}
])
< {
  _id: 'M2',
  totalTicketsSold: 3,
  totalRevenue: 1050,
  averagePrice: 350,
  minimumPrice: 350,
  maximumPrice: 350
}
{
  _id: 'M1',
  totalTicketsSold: 3,
  totalRevenue: 900,
  averagePrice: 300,
  minimumPrice: 300,
  maximumPrice: 300
}
{
  _id: 'M3',
  totalTicketsSold: 2,
  totalRevenue: 800,
  averagePrice: 400,
  minimumPrice: 400,
  maximumPrice: 400
}
```

Query 3: Customer-wise spending report

Goal:

Find how much each customer has spent and which movies they watched.

Result:

For each customer, display:

- number of movies booked
- total amount spent
- average spending
- list of movies watched

```
> db.bookings.aggregate([
  {$lookup:{
    from: "movies",
    localField: "movie_id",
    foreignField: "_id",
    as: "movieDetails"}}},
  {$unwind: "$movieDetails" },
  {$lookup:{
    from: "customers",
    localField: "customer_id",
    foreignField: "_id",
    as: "customerDetails"
  }},
  { $unwind: "$customerDetails" },
  { $group: {
    _id: "$customer_id",
    customerName: { $first: "$customerDetails.name" },
    numberOfMoviesBooked: { $sum: 1 },
    totalAmountSpent: { $sum: "$movieDetails.price" },
    averageSpending: { $avg: "$movieDetails.price" },
    moviesWatched: { $push: "$movieDetails.title" }
  }},
  { $sort: { totalAmountSpent: -1 } }
])
```


Query 4: Customers with multiple bookings

Goal:

Identify customers who booked more than one movie.

Result:

List of customers who made multiple bookings.

```
> db.bookings.aggregate([
  {$lookup:{
    from: "customers",
    localField: "customer_id",
    foreignField: "_id",
    as: "customerDetails"
  }},
  {$unwind:"$customerDetails"},
  {$group:{
    _id: "$customer_id",
    customerName:{$first:"$customerDetails.name"},
    totalBookings:{$sum:1}
  }},
  {$match:{totalBookings:{$gt:1}}},
  {$sort:{totalBookings: -1}}
])
< {
  _id: 3,
  customerName: 'Rahul',
  totalBookings: 2
}
{
  _id: 1,
  customerName: 'Arun',
  totalBookings: 2
}
{
  _id: 5,
  customerName: 'Kiran',
  totalBookings: 2
}
```

Query 5: Genre-wise booking count

Goal:

Find which movie genre is most popular.

Result: For each genre, show the total number of bookings.

```
> db.bookings.aggregate([
  { $lookup: {
    from: "movies",
    localField: "movie_id",
    foreignField: "_id",
    as: "movieDetails"
  }},
  { $unwind: "$movieDetails" },
  { $group: { _id: "$movieDetails.genre", totalBookings: { $sum: 1 } }},
  { $sort: { totalBookings: -1 } }
])
< {
  _id: 'Sci-Fi',
  totalBookings: 5
}
{
  _id: 'Fantasy',
  totalBookings: 3
}
```

Query 6: Movie popularity with customer list

Goal:

Show movie popularity along with customer details.

Result:

For each movie, display:

- total number of bookings and list of customers who booked the movie

```
> db.bookings.aggregate([
  { $lookup: {
    from: "customers",
    localField: "customer_id",
    foreignField: "_id",
    as: "customerDetails"
  }},
  { $unwind: "$customerDetails" },
  { $group: {
    _id: "$movie_id",
    totalBookings: { $sum: 1 },
    customerList: { $push: "$customerDetails.name" }
  }},
  { $sort: { totalBookings: -1 } }
])
```

```
< {
  _id: 'M2',
  totalBookings: 3,
  customerList: [
    'Arun',
    'Neha',
    'Kiran'
  ]
}
{
  _id: 'M1',
  totalBookings: 3,
  customerList: [
    'Arun',
    'Priya',
    'Rahul'
  ]
}
{
  _id: 'M3',
  totalBookings: 2,
  customerList: [
    'Rahul',
    'Kiran'
  ]
}
```